



SUPPLIWISE

SuppliWise

Developer Manual

Architecture, request lifecycles, data model and extension playbooks.

For engineers maintaining and extending the system

SuppliWise · Developer Manual · Version 1.0

Wellness guidance, not a diagnosis. Always consult a qualified healthcare professional before starting, stopping or changing any supplement or medication.

SuppliWise — Developer Manual

Architecture, request lifecycles, data model, and extension playbooks.

This is the technical companion to [README.md](#) . For the end-user guide see [docs/USER_MANUAL.md](#) . For the feature → code map of the blockchain layer see [BLOCKCHAIN_FEATURES.md](#) . For the security audit see [SECURITY_AUDIT_REPORT.md](#) .

Table of contents

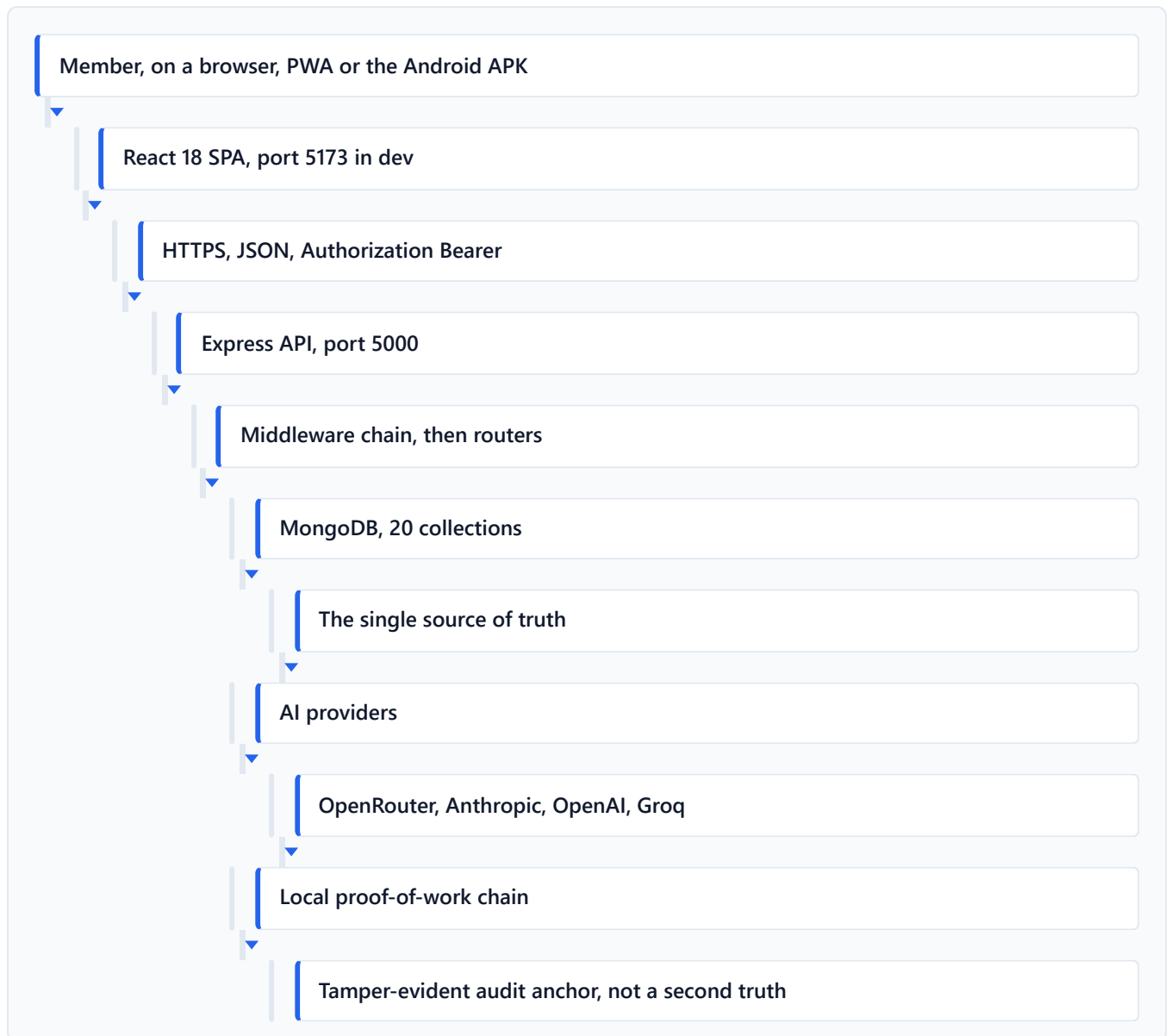
1. System architecture
 2. Repository layout
 3. The request lifecycle
 4. Authentication and sessions
 5. Data model
 6. Entitlements and plans
 7. The AI layer
 8. The assessment → recommendation pipeline
 9. Plan day: the 4 AM rule
 10. The blockchain layer
 11. Rate limiting and abuse control
 12. Testing
 13. Extension playbooks
 14. Operations and troubleshooting
-

1. System architecture

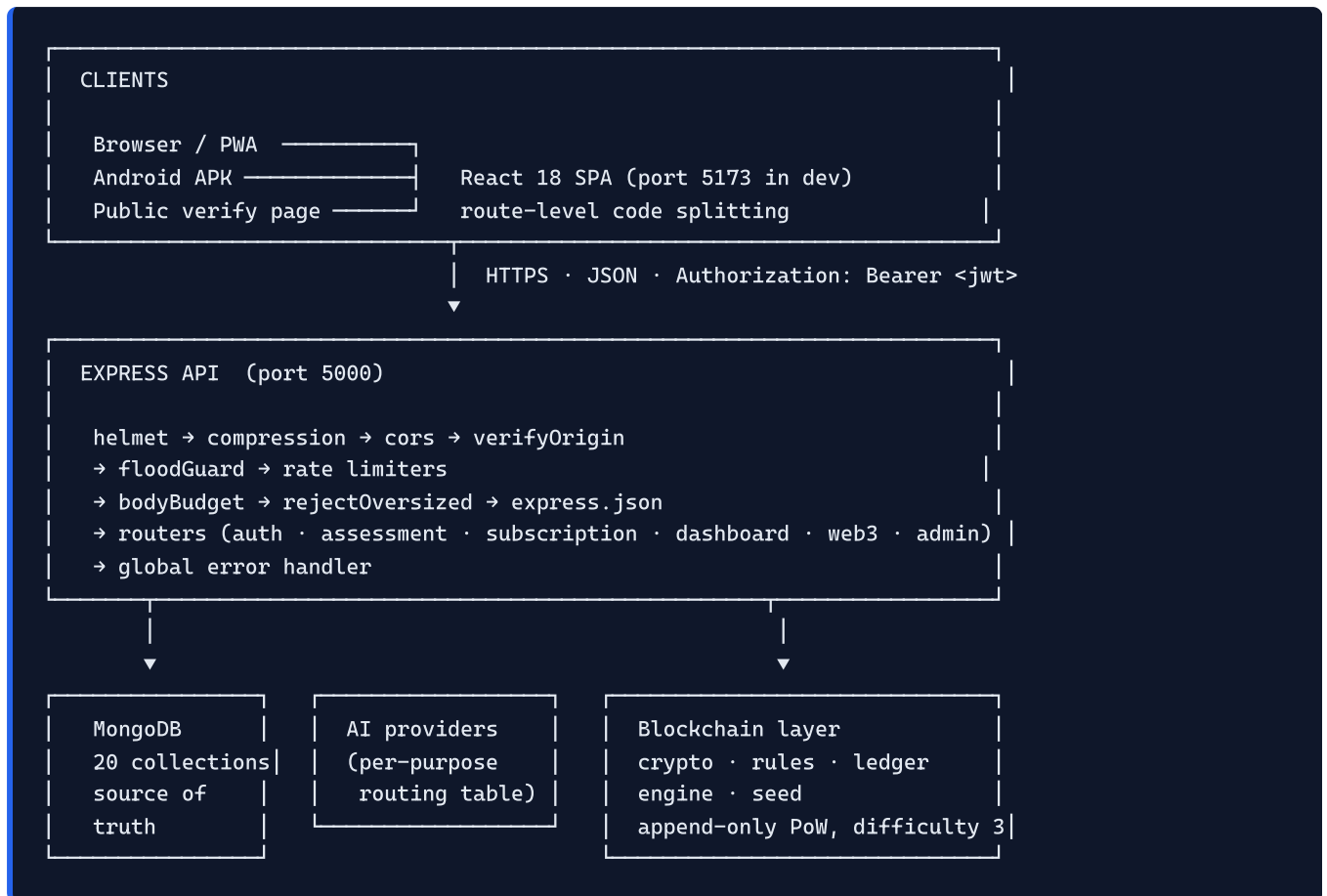
1.1 The one-paragraph version

A React 18 + Vite single-page app talks to an Express 4 API over HTTP/JSON. MongoDB via Mongoose is the single source of truth. AI work is dispatched through one routing table to several providers. A self-contained proof-of-work chain inside the server acts as a tamper-evident audit anchor. Entitlements are resolved on the server and mirrored on the client purely so gates can render instantly.

1.2 Component map



1.3 Runtime topology



1.4 Two properties that shape everything else

MongoDB is the source of truth. The chain is an audit anchor — never a second source of truth. The contract in `server/blockchain/engine.js` is: *mutate the database first, then anchor the change; an anchor failure is logged and never fails or rolls back the business operation.* Availability beats perfect atomicity.

The server is the single source of truth for entitlements. The frontend mirror (`my-react-app/src/subscription/features.js`) exists only so gates can render instantly without a round trip. A contract test fails if the two drift.

2. Repository layout

```
SuppliWise/
├── server/                                Express API
│   ├── index.js                          app, middleware order, rate limiters, boot
│   ├── models/                           20 Mongoose modules
│   │   ├── User.js Assessment.js IntakeRecord.js DashboardMetrics.js
│   │   ├── AdminAccount.js Session.js SecurityEvent.js AdminEvent.js
│   │   ├── UserNotification.js Passkey.js BackupCode.js AuthChallenge.js
│   │   ├── MfaTransaction.js PasswordResetToken.js SupplementDetail.js
│   │   ├── SubscriptionRequest.js SubscriptionCancelRequest.js
│   │   ├── ChatThread.js ChatMessage.js
│   │   └── Web3.js                       20 Web3 schemas + DEFAULT_PARAMS
│   ├── routes/                           HTTP surface
│   │   ├── auth.js passwordReset.js passkeys.js totp.js
│   │   ├── recoveryCodes.js sessions.js assessment.js subscription.js
│   │   ├── recommend.js chat.js polish.js supplement_detail.js
│   │   ├── dashboard.js insights.js notifications.js security.js
│   │   ├── supportChat.js adminSupportChats.js admin.js
│   │   ├── securityRedeem.js             attached to security.js, public
│   │   └── web3/                         chain supply rewards market govern data
│   │                                       ecosystem guards index
│   ├── middleware/
│   │   ├── auth.js                       protect · adminOnly · session cache
│   │   └── stepUp.js                     requireStepUp · hasStepUp · issue
│   ├── blockchain/
│   │   ├── crypto.js                     stableStringify · sha256 · ed25519 · AES-GCM
│   │   ├── rules.js                     PURE decision logic (no I/O)
│   │   ├── ledger.js                    PoW append / verify / audit
│   │   ├── engine.js                    the ONLY state mutator
│   │   └── seed.js                       idempotent bootstrap
│   ├── utils/                           62 modules (see §7, §9, §11)
│   ├── scripts/                         check-syntax · notify-admins
│   │                                       send-admin-credentials
│   ├── tools/patch.js                    one-shot verified replacement
│   ├── Test File/                       63 *.test.js · node:test
│   └── test-*.js                         HTTP smoke suites
├── my-react-app/                         React 18 + Vite 8
│   ├── vite.config.js                    PWA, code splitting, optimizeDeps
│   ├── scripts/                          verifyDevDeps · healDepsCache
│   │                                       verifyAdminUserIsolation
│   └── src/
│       ├── App.jsx                       router + guards
│       ├── main.jsx api.js
│       ├── Pages/                         26 page components
│       ├── Components/                   Navbar · Web3Panels · AdminUI · ...
│       ├── subscription/                  features.js · catalogue.js · paymentCopy.js
│       ├── utils/                         slotSchedule.js · nameValidation.js
│       └── api/web3.js                     thin Web3 client
```

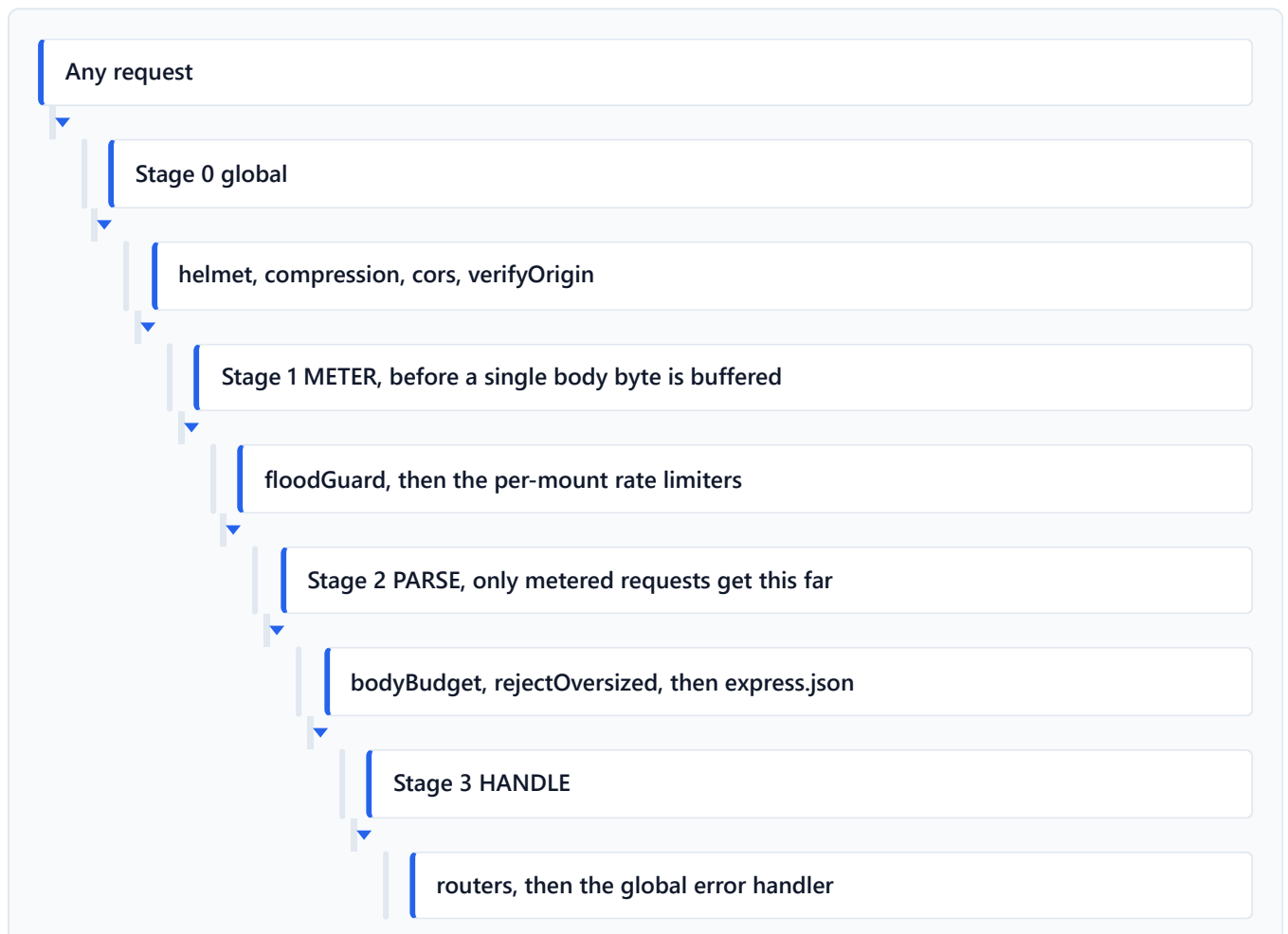
2.1 Why `rules.js` is separate from `engine.js`

Every "smart contract" decision — quorum tallies, streak payouts, APY pro-rating, escrow fee splits, dispute outcomes — is a **pure function with no I/O**. That makes them exhaustively unit-testable without a database, which is exactly what `Test File/web3.test.js` does. `engine.js` owns persistence and anchoring; `rules.js` owns decisions.

3. The request lifecycle

`server/index.js` mounts middleware in a deliberate three-stage order. **The order is the design** — changing it breaks guarantees.

3.0 The three stages



3.1 Stage 0 — global, before anything

Order	Mount	Middleware	Why here
1	global	<code>helmet</code> (CSP)	JSON-only origin
2	global	<code>compression</code>	honours <code>x-no-compression</code>
3	global	<code>cors</code>	shares one allowlist with the CSRF guard; never a wildcard with credentials
4	global	<code>verifyOrigin</code>	CSRF / server-side origin rule on state-changing requests
5	<code>/api</code>	<code>Cache-Control: no-store</code>	
6	<code>/pictures</code>	static, outside <code>/api</code>	served ahead of all limiters; CORP overridden to <code>cross-origin</code>

`express.json()` is deliberately NOT here. It is mounted in stage 2, so body metering always runs ahead of parsing. Putting it earlier would let an unmetered body be buffered.

3.2 Stage 1 — meter (before a single body byte is buffered)

Mount	Limiter
<code>/api</code>	<code>requestFloodGuard</code> — global per-IP envelope, meters but never escalates
<code>/api/auth/me</code>	<code>sessionLimiter</code>
<code>/api/auth</code>	<code>lockoutCheck</code> + <code>authLimiter</code>
<code>/api/admin</code>	<code>lockoutCheck</code> + <code>adminLimiter</code>
<code>/api/recommend</code>	<code>recommendLimiter</code> (protects AI quota)
<code>/api/assessment</code> <code>/api/chat</code> <code>/api/support-chat</code> <code>/api/dashboard</code> <code>/api/insights</code> <code>/api/notifications</code> <code>/api/security</code> <code>/api/web3</code>	<code>userLimiter</code>
<code>/api/polish</code> <code>/api/supplement-detail</code>	<code>aiLimiter</code>
<code>/api/subscription</code>	<code>sessionLimiter</code>

3.3 Stage 2 — parse (only metered requests get this far)

```
bodyBudget(10 MiB)      global, reserves in-flight bytes before parsing
rejectOversized(10 MiB) /api/auth/profile · /api/admin/profile
                        /api/subscription/requests
rejectOversized(1 MiB)  global blanket
express.json({ limit }) 10 MiB on the three upload paths, 1 MiB elsewhere
```

3.4 Stage 3 — handle

Routers mount in a fixed order — auth sub-routers **before** the general `auth` router so their paths win. Then the global error handler, then `GET /api/health`, then the JSON 404.

3.5 Error handling contract

Condition	Response
<code>err.type === 'entity.parse.failed'</code>	<code>400 { message: 'Invalid JSON request body.' }</code> , no parser internals logged
<code>4xx</code>	<code>logClientError(status, method, path, message)</code> — throttled, never a full stack
<code>5xx</code>	full <code>console.error</code> with stack, path, method
Client message	<code>err.message</code> below 500, else a generic string

A flood of `413`s must never be able to DoS the log, which is why 4xx logging is throttled.

3.6 Crash guards

Event	Behaviour
<code>unhandledRejection</code>	log only — a bad request must never take the API down
<code>uncaughtException</code>	fatal only for <code>EADDRINUSE</code> / <code>EACCES</code> / <code>EADDRNOTAVAIL</code> (no server left to keep serving); otherwise log and continue

Fix the route, not the guard.

4. Authentication and sessions

4.1 Member sign-in

```
POST /api/auth/login
├─ lockoutCheck + authLimiter
├─ Load the user by email
│   └─ Unknown user?
│       ├── Burn a throwaway argon2 compare so the timing matches
│       └─ Return the IDENTICAL body a wrong password produces
│   └─ Verify the password (argon2id, or bcrypt then transparently upgrade)
│       ├── Wrong? → Record the offence, run the escalating ladder
│       └─ Correct? → Check ban / inactive state (AFTER the password proof,
│           │                                     never before it)
│               ├── Banned or inactive → refuse
│               └─ Active → Check 2FA
│                   ├── 2FA ON → mint an MfaTransaction
│                   │           (single use, attempt budget)
│                   │       ├── Valid → completeSignIn
│                   │       └─ Invalid → decrement the attempt budget
│                   └─ 2FA OFF → completeSignIn
│                       └─ Issue the Session row, set User.currentSessionId,
│                           stamp sid into the JWT, write the SecurityEvent,
│                           return publicUser
```

Three properties worth preserving:

- 1. Enumeration-safe.** An unknown email and a wrong password produce byte-identical responses *and* near-identical timing — the unknown path burns a real password compare.
- 2. Ban/inactive checks run AFTER the password proof.** Checking earlier turns the login form into an unauthenticated oracle for which emails are banned.
- 3. One active session per account,** enforced atomically via `User.currentSessionId`. A new sign-in revokes only the displaced session *of the same account*, so multi-account use across tabs is unaffected.

4.2 Session policy

Surface	Expiry
Member sessions	Never. No <code>exp</code> claim, no idle timer. You are signed out only by signing out, by a newer sign-in on that account, by a password reset, or by the account being disabled.
Admin sessions	Yes. <code>ADMIN_IDLE_MINUTES</code> (default 10) idle window; <code>ADMIN_TOKEN_LIFETIME_MINUTES</code> (default 15).

`Session.expiresAt` exists purely as an operator lever for member sessions.

The server **refuses to boot** if `ADMIN_TOKEN_LIFETIME_MINUTES <= ADMIN_IDLE_MINUTES` — that ordering logs a working administrator out mid-task.

4.3 The `protect` middleware

```
Authorization: Bearer <jwt>           (header only – query-string tokens are REJECTED)
jwt.verify(token, secret, { algorithms: ['HS256'] })  pinned, never "any"
role === 'admin'
  AdminAccount exists and enabled?
  No --> 401
  Idle window exceeded (fails CLOSED on a null stamp)? --> 401
  mustChangePassword set and path not on the allowlist? --> 403 PASSWORD_CHANGE_REQUIRED
  Otherwise --> heartbeatAdmin() unless x-admin-background
role === 'user'
  verifyUserSession(decoded)  30 s / 5000-entry cache
  Session missing or lookup failed --> 503 (retryable), not 401
  Revoked / not current / expired --> 401
  Account inactive --> 401
  Otherwise --> attach req.user, req.sessionId
```

4.4 Step-up authentication

Possessing a session is **not** enough to change security settings.

User clicks Change Password under Account Security

POST /api/security/step-up with password and current TOTP

Issue a 5-minute JWT carrying sub, id, sid, purpose

Client returns it in the X-Step-Up header, never in a URL

`requireStepUp` verifies signature, expiry and purpose

Confirms `claims.id` equals `req.user._id`

Confirms `claims.sid` equals `req.sessionId`

Any failure returns 401 STEP_UP_REQUIRED, same message every time

The identical-failure-message rule is deliberate: a distinct error per failure mode turns the endpoint into a probe.

4.5 The escalating lockdown ladder

```
offence 1 --> 15 minutes
offence 2 --> 1 hour
offence 3 --> 6 hours
offence 4 --> 24 hours
offences decay after 30 days; a separate 3-strike account counter runs alongside
```

The **2FA ladder is capped at 1 hour** on purpose — otherwise an attacker could lock a victim out of their own recovery path indefinitely.

4.6 Password policy — one source

`server/utils/passwordRules.js` : **10–128 characters**, upper + lower + digit + symbol, plus a blocklist of common passwords, decorated variants, sequences and repeated runs. It also refuses passwords built from the account's email local part. `describePasswordRules()` is served **over the wire** so the client checklist cannot drift, and a contract test fails if it does.

5. Data model

20 collections. Highlights and the indexes that matter.

5.1 Identity and auth

Model	Purpose	Key indexes
User	identity, argon2id hash, profile/banner pictures, <code>twoFactorMethod</code> + sealed seed, <code>currentSessionId</code> , <code>sessionVersion</code>	<code>{ createdAt: -1 }</code> , <code>{ lastLoginAt: -1 }</code> , <code>{ subscriptionPlan: 1, subscriptionExpiresAt: 1 }</code>
AdminAccount	admin identity; <code>passwordHash</code> and <code>totpSecret</code> are <code>select: false</code>	<code>alias</code> unique
Session	one per sign-in; <code>_id</code> IS the <code>sid</code> in the JWT	<code>{ user, createdAt }</code> , <code>{ user, revokedAt }</code>
Passkey	one WebAuthn credential. No private key is ever stored	<code>{ user, createdAt }</code>
AuthChallenge	single-use WebAuthn challenge (a DB row, so it cannot be replayed)	<code>expiresAt</code> TTL 0
MfaTransaction	the persisted <code>MFA_REQUIRED</code> state	<code>{ user, consumedAt, expiresAt }</code> , <code>expiresAt</code> TTL
BackupCode	single-use recovery codes, SHA-256 digests only	<code>{ user, batchId }</code> , <code>{ user, usedAt }</code>
PasswordResetToken	emailed grant; <code>tokenHash</code> / <code>codeHash</code> are <code>select: false</code>	<code>tokenHash</code> unique, <code>expiresAt</code> TTL

Single-use is enforced by a **conditional update on** `usedAt: null`, not a read-then-write. Two concurrent redemptions cannot both win.

5.2 Health and tracking

Model	Purpose
Assessment	every field plus a full <code>aiResults</code> snapshot (<code>Mixed</code>), <code>wellnessBaseline</code> , priority flag, 5-year <code>expiresAt</code>
IntakeRecord	one dose per plan day: <code>dayKey</code> , taken, timestamps. Indexed <code>{ user, assessment, dayKey }</code> and <code>{ user, dayKey }</code>
DashboardMetrics	per-assessment streak, adherence, wellness score
SupplementDetail	cached AI guide, generic <code>detail</code> + <code>personalized</code> keyed by profile digest. <code>nameKey</code> unique
UserNotification	member inbox rows, type enum <code>['severe-flag', 'info']</code>

`aiResults` being `Mixed` is why there are **no migrations** in this project.

5.3 Billing, support, audit

Model	Purpose
<code>SubscriptionRequest</code> / <code>SubscriptionCancelRequest</code>	the two admin review queues; amounts are server-side pricing snapshots
<code>ChatThread</code> / <code>ChatMessage</code>	stored support conversations
<code>SecurityEvent</code>	append-only , closed enum of ~35 event types, fixed metadata allowlist
<code>AdminEvent</code>	admin audit trail

`SecurityEvent` has **no free-form metadata field on purpose** — otherwise a secret could be smuggled into an event and the "we never log secrets" claim would be false.

5.4 Cross-cutting behaviours

- Pre-save hooks stamp `passwordChangedAt` in the **same write** as the new hash, so the two can never disagree.
- Deleting a user cascades to assessments, intake records and dashboard metrics.
- Recoverable state (e.g. `resolveFeatureKey`) uses prototype-less lookups and **fails closed**.

6. Entitlements and plans

6.1 Plan vocabulary — defined exactly once

`server/utils/subscriptionState.js` is a **leaf module** (requires nothing from the codebase) and is the single definition of the plan ids. It is re-exported by `entitlements.js`, `planCatalogue.js` and the deprecated `plan.js` shim, and mirrored for the UI in `my-react-app/src/subscription/features.js`.

```
PLAN_RANK   free:0  monthly:1  annual:2  custom:3
PLAN_ORDER  ['free', 'monthly', 'annual', 'custom']
PLAN_LABELS free:FREE  monthly:DELUXE  annual:PREMIUM  custom:ULTIMATE
```

The stored ids are the contract and are **never renamed** — only the display labels change. A contract test fails if the server and client registries drift.

6.2 The feature matrix

`minTier` is inclusive.

Feature key	minTier	Label
healthAssessment	free	Health Assessment
recommendations	free	Supplement Recommendations
dailyIntake	free	Daily Intake
insights	monthly	Insights & Analytics
pdfExport	monthly	PDF Report Export
web3	monthly	Web3 Wallet & Supply Chain
market	monthly	Marketplace
dao	monthly	DAO Governance
priorityAssessment	annual	Priority Assessment
historyFull	annual	5-Year Record History
chat	custom	AI Chat Assistant

6.3 Billing rules

```

STANDARD_PERIOD_DAYS = 30      a purchase is exactly 30 days, nothing auto-renews
repurchase                     --> EXTENDS the window (never restarts, never discards a paid day)
plan change                    --> applies immediately, no sign-out
expiry                         --> resolved at READ time, no cron
                                a past subscriptionExpiresAt reads as FREE
                                subscriptionPermanent === true ignores the stored expiry
days remaining                --> Free 5 / Deluxe 10 / Premium 20 / Ultimate 20 per page

```

There is a fifth "Team" band in an older README revision. It was **deliberately removed**: it was priced per seat but granted exactly what Premium granted, it was not a real tier (no `team` plan id existed), and no account could ever hold a second seat. `plan-catalogue.test.js` enforces its absence. Do not reintroduce it.

6.4 Fail-closed lookups

```

getFeature(key) {
  if (!Object.prototype.hasOwnProperty.call(FEATURES, key)) return null;
  return FEATURES[key];
}

```

`constructor`, `__proto__` and `toString` resolve to nothing, so every gate answers "locked". Without `hasOwnProperty`, `FEATURES['constructor']` would return `Object` and `can()` would throw.

6.5 How a plan change reaches the UI

```
Admin approves a request
server updates the subscription record
subscriptionBus.publish(state)
  invalidateUserSessionCache(userId)    the entitlement cache is keyed by TOKEN,
                                         so an in-place account switch can never
                                         hand one account's plan to the next
every open SSE stream at /api/subscription/stream receives the new state
client entitlement store updates
  a focus refresh catches a tab that was asleep
  an expiry watch handles natural lapse
  a BroadcastChannel relays to sibling tabs
```

7. The AI layer

7.1 One routing table

`server/utils/aiRouter.js` holds `AI_ROUTES`. **No route hard-codes a vendor.** That is not tidiness — each call site used to carry its own URL and model string and they had already drifted: `chat.js` and `polish.js` sent `deepseek/deepseek-v4-flash` while the admin panel reported `deepseek/deepseek-v4-flash-0731` for the same key. OpenRouter's probe cannot catch this (its model list is ~752 KB and will not fit the 5 s probe budget), so it would have surfaced only as a failed user request.

Purpose	Provider	Fallback	Default model
<code>assessment</code>	OpenRouter (default)	—	<code>deepseek/deepseek-v4-flash-0731</code>
<code>chat</code>	Anthropic	OpenRouter	<code>claude-haiku-4-5-20251001</code>
<code>polish</code>	OpenAI	OpenRouter	<code>gpt-5.4-nano</code>
<code>supplementDetail</code>	OpenRouter (default)	—	<code>deepseek/deepseek-v4-flash-0731</code>
<code>systemDetection</code>	Groq	—	<code>openai/gpt-oss-120b</code>
<code>priorityFlagging</code>	Anthropic	—	<code>claude-haiku-4-5-20251001</code>

The split is by workload shape: member-facing health content on OpenRouter, the busiest and shortest call (`polish`) on its own cheap provider, latency-sensitive scheduled work on Groq, and chat + clinical second opinion on Anthropic.

To change the split, edit `AI_ROUTES`. Nothing else needs to know.

7.2 Fallback vs. default substitution — two different things

```
resolveChain(purpose)
```

1. primary = the purpose's own provider
2. if a fallback is DECLARED and differs, resolve it and append
3. drop anything not configured
4. if nothing survives, substitute the default provider, tagged `viaDefault: true`

	Declared fallback	Default substitution
Triggered by	the primary genuinely cannot answer	the primary is entirely unconfigured
Reached when	no key, rejected, credit exhausted, timeout, empty reply	at configuration time
Tagged	no	<code>viaDefault: true</code>

`chat` needs a fallback because a key can be perfectly valid and still unable to serve a completion — Anthropic answers an exhausted balance as a **400**, which the model-list health probe cannot see. Moving chat to an unfunded account would otherwise put every user on the canned offline reply.

7.3 Budget splitting

```
DEFAULT_TIMEOUT_MS = 15000    PRIMARY_BUDGET_FLOOR_MS = 5000

splitTimeout(remaining, totalMs)
  remaining <= 1      --> the caller's whole budget
  otherwise           --> max(5000, floor(total / remaining))
```

The primary can never spend the whole budget while a fallback waits; the last provider keeps whatever is left.

7.4 `completeWithTarget` never throws

Every failure becomes a value.

Failure	Reported as
HTTP 401 / 403	<code>Key rejected (HTTP N) – rotate <ENV_KEY>.</code>
other non-2xx	credit-exhausted detection first ("add credit"), else <code>Provider answered HTTP N.</code>
abort	<code>Request failed – timed out after N ms.</code>
empty completion	<code>Provider returned an empty completion.</code>
JSON mode but not an object	<code>Provider reply was not a JSON object.</code>
400 with <code>json: true</code>	retried once without <code>response_format</code>

Key material is never returned. `resolveTarget` hands the key to the caller for one request and never copies it into a result.

7.5 Anthropic is not OpenAI-shaped

Anthropic's Messages API takes `system` as a **top-level field**, has no `role: 'system'` message, and **rejects adjacent same-role turns**. The router builds a different body per provider (`wire: 'anthropic'`):

```
Anthropic wire
  lift `system` out of the conversation into a top-level field
  merge adjacent same-role turns
  drop a leading assistant turn
  read the different response shape
OpenAI wire
  pass reasoning.effort etc. via extraBody
```

`extraBody` may be an object **or a function of the target**, so gateway-only fields can be scoped by provider key — Anthropic answers an unknown top-level key with a 400.

7.6 Priority flagging — escalate-only by design

```
POST /api/assessment
analyzeSeverity(submission)      RULE engine, synchronous, pure, UNMODIFIED
  fixed lists of red-flag symptoms, critical conditions, emergency phrases
  -> { flagged, reasons }          auditable, instant, un-talkable-around
if rules did NOT flag:
  priorityFlagging.analyzePriorityFlagging(submission)  Anthropic second opinion
  requires ALL of: rules did not flag AND confidence >= 80 AND >= 1 citable reason
  ANY failure (no key, no credit, rejected, timeout, unparseable, schema-invalid)
    --> leave the rule verdict COMPLETELY untouched
final = ruleVerdict.flagged || aiEscalated
```

The AI can add a flag and add reasons. It can never remove one. The asymmetry is deliberate: a missed severe case is a clinical risk nobody reviews, while a spurious flag pauses a user's new assessments — a real cost this codebase has already been bitten by (see `priorityGate.js`).

Header note worth preserving: this layer once flagged nearly everything, because the word "severe" in a generic AI disclaimer matched.

Privacy: only health fields are sent — never a name, email or picture. Patient free text is stripped of emails, phone numbers and handles, and instruction-override phrases are defanged, because `feelingDescription` is written by whoever filled in the form.

7.7 System detection

`utils/systemDetection.js` reads Security Center probe results and predicts what is likely to break next. Two properties matter more than the prediction:

- **It cannot break detection.** Every failure path falls back to a rule-based verdict from the same probes. A security panel that goes dark when a third-party API is down is worse than one that never predicted anything.
- **Model output is never trusted.** Severity is clamped to `nominal | watch | elevated | severe`, categories to a fixed vocabulary, strings length-bounded, and a malformed reply is discarded whole rather than half-applied.

Probe details are redacted before leaving the process — emails, IPs, JWTs, API keys and long hashes are replaced.

8. The assessment → recommendation pipeline

```
POST /api/assessment
├─ Validate and sanitise every free-text field
│   └─ spam / garbage input stripped, unreadable fields reported back
├─ Compute severity with the RULE engine
│   └─ flagged? → Priority gate: new assessments pause until reviewed
├─ Persist the Assessment document
└─ aiRouter.complete('assessment', prompt)
    ├─ 1. provider returns a JSON wellness plan
    ├─ 2. sanitizeStrings() cleans the AI output recursively
    ├─ 3. rule-based clinical fallback if EVERY provider is unreachable
    └─ Persist as Assessment.aiResults (Mixed)
        20 supplements sorted High → Medium → Low, then confidence
        daily schedule · meals · lifestyle · 5-phase action plan
            └─ client renders /results
```

8.1 Always produces a result

1. DeepSeek V4 Flash via OpenRouter	full JSON wellness plan, 20 supplements
2. <code>sanitizeStrings()</code>	recursive clean of AI output
3. Rule-based clinical fallback	covers all age groups, medications, conditions, symptoms, diet, lifestyle, pregnancy

If **every** AI provider is unreachable, step 3 still returns a complete plan. That is why the feature never shows an empty state after a successful submit.

8.2 Prompt quality rules encoded in the prompt

Rule	Constraint
4	consider only provided information — never invent symptoms or severity
9	severity labels only for symptoms actually reported
13	<code>triggeredBy</code> uses only reported data — falls back to "General wellness"
—	<code>conditionContext</code> returns <code>null</code> when there is nothing to reference
—	meal rules strictly respect diet type, allergies and medical conditions

8.3 Supplement detail caching

`/api/supplement-detail` is assessment-aware and cached in `localStorage` by `assessmentId + supplementName` with a **30-day expiry**, so re-reading a card costs nothing.

9. Plan day: the 4 AM rule

`server/utils/planDay.js` owns the only `Intl` reader in the server and is **the one function every caller should use**.

```
planDayKey(now, timeZone)
  parts = zonedNowParts(now, timeZone)
  if (!parts) return planDayKey(new Date(), FALLBACK_TIME_ZONE)
  if (parts.minutes >= 240) return today           in the user's own zone
  return previousDayKey(today)                   still yesterday's plan day
```

9.1 Why it exists

Every handler used to compute "today" itself from `new Date().toISOString()`, which rolls at **00:00 UTC** — 8:00 AM in Manila, 7:00 PM the previous evening in Los Angeles. The consequences were all user-visible:

- a plan displayed for four hours after the reset it was meant to have
- the edits that display invited refused as "only today's supplements"
- the weekly chart's newest column always empty for half the planet
- the streak evaluated against a day the user could not see

There were **three copies on the server plus a fourth** local-midnight copy in the tracker. Four copies of one boundary is four chances to disagree.

9.2 Invariants to preserve

- **The clock cannot be faked from a header.** `X-Client-Timezone` is the legitimate lever and only selects a *zone*; it can never set the instant.
- `isInDeadHours()` **fails open** (returns `false` on an unusable clock) — failing closed would penalise a user for the four hours in which nothing may be recorded.
- The server's window table in `utils/intakeWindows.js` deliberately duplicates the client's `src/utils/slotSchedule.js` (CJS vs ESM, kept in step by hand). Each side asserts itself.
- A dose taken **late** is not "missed"; a dose inside an **open window** is never charged.

9.3 Testing the boundary

`Test File/plan-day-routes.test.js` puts the server's own instant on either side of the boundary using `Etc/GMT±N` offsets.

Use an adjacent pair. `zonesEitherSideOfReset()` returns the *highest* offset on each side, and those two can straddle the international date line — at 10:00 UTC the highest dead-hours zone (`Etc/GMT-14`) reads 00:00 on the 5th, giving plan day the 4th, while the highest after-reset zone (`Etc/GMT-13`) reads 23:00 on the 4th, also the 4th. They agree, correctly. Asserting they "must disagree" tests the offset grid, not the code, and failed for 4 hours out of every 24. `adjacentResetPair()` picks two zones one hour apart, so neither local date rolls between them and the plan days are provably one day apart.

10. The blockchain layer

10.1 What it is, and what it is not

- MongoDB is the source of truth for balances and business records.
- The chain is a **tamper-evident audit anchor**, not a second source of truth.
- **No external chain, no gas, no wallets to fund, no RPC endpoint.**

10.2 The privacy rule

Personal data is **never** written on-chain. Anything that could contain personal data is passed to `engine.anchor(type, actor, { secret })`, which stores only `sha256(stableStringify(secret))` as the transaction's `dataHash`. `stableStringify` sorts keys recursively, so the same payload always produces the same hash.

10.3 Block and transaction shape

```
Block { index (unique), timestamp, prevHash, nonce, hash, txs: [Tx] }
Tx    { txHash, type, actor, data /* public metadata only */, dataHash /* always present */,
      timestamp }

normalizeTx(tx) --> hash the tx fields + its dataHash --> txHash
headerHash(hdr) --> hash index, timestamp, prevHash, nonce, canonical txs --> hash
append(txs)     --> mine a nonce until hash has DIFFICULTY leading zero hex chars,
                  then insert with prevHash = previous block's hash
verify()        --> recompute EVERY header hash AND every prevHash link
```

Difficulty `3` (`CHAIN_DIFFICULTY` , clamped 1–5). Raising it slows every anchor — the smoke suite issues ~90 anchors per run, so keep it at 3 locally.

10.4 `verify()` has three paths

Path	Use	Cost
<code>audit({ maxAgeMs })</code>	deep O(chain) re-hash. Shares one in-flight walk across concurrent callers	expensive
<code>verifyTail()</code>	fast path — re-hashes only blocks past the checkpoint. Falls back to a full audit on a gap or any break	cheap
<code>verify({ maxAgeMs, budgetMs })</code>	request path. Caches the promise so concurrent callers share one walk. On budget expiry returns <code>{ valid: true, pending: true }</code>	bounded

On budget expiry the answer is **"unknown", never "broken"** (`valid: true, pending: true`). Reporting a timeout as a failure would page an operator for nothing.

A checkpoint is persisted in `SwChainAudit` ; only a deep audit advances its `verifiedAt` .

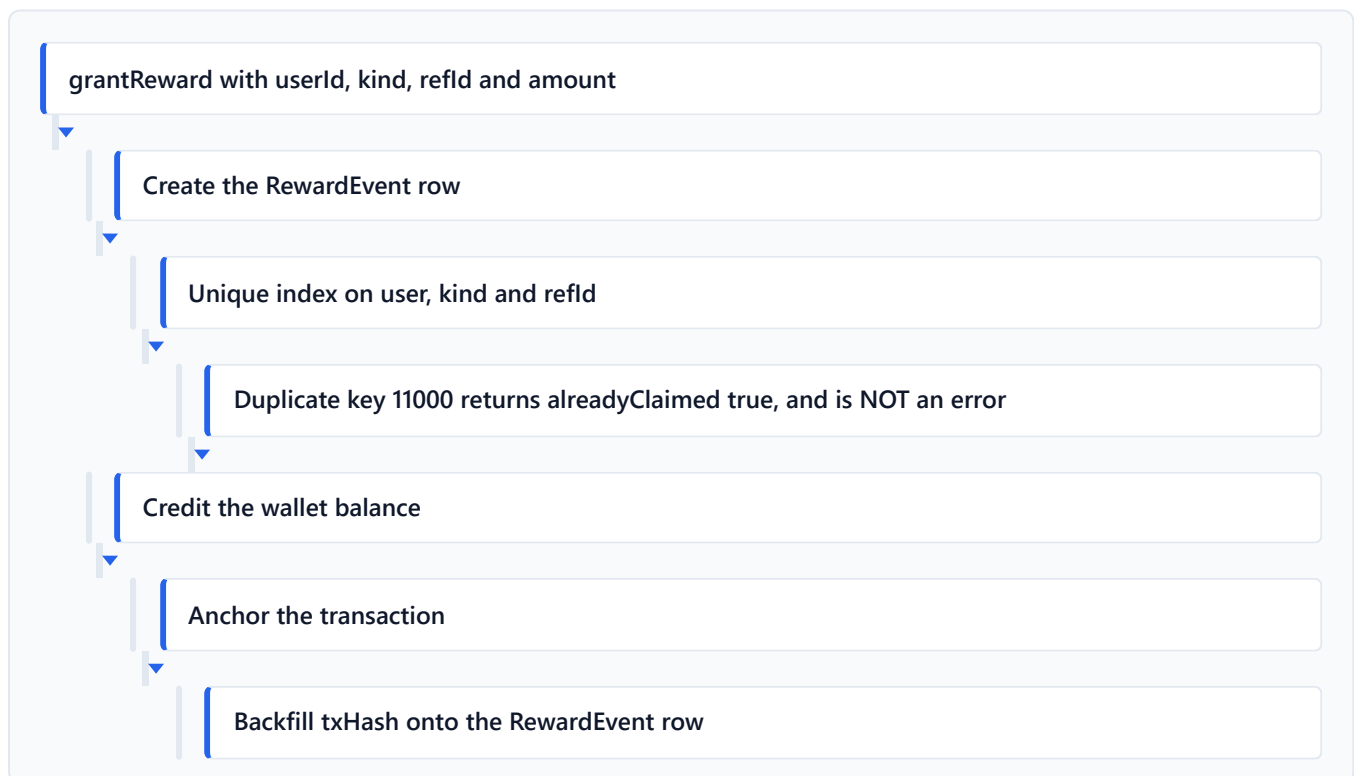
10.5 The state engine — the only mutator

Routes **never** touch `Wallet` balances directly.

Function	Contract
<code>getConfig()</code> / <code>setParam()</code>	DAO params, cached 30 s, backfilled from <code>DEFAULT_PARAMS</code>
<code>ensureWalletForUser()</code>	Creates wallet + DID + key; grants the welcome bonus through <code>grantReward</code> so history is complete from block 0. Safe under a concurrent first hit (unique-index race resolved)
<code>credit()</code> / <code>debit()</code>	One-sided + anchor. <code>debit</code> is conditional on sufficient funds
<code>transfer()</code>	Atomic two-sided. A failed credit rolls the debit back before rethrowing
<code>stake()</code> / <code>unstake()</code>	Move balance $\Rightarrow$ staked, accruing pending yield first
<code>accrueStake()</code>	Pro-rated APY guarded by a conditional update on the exact last-accrual timestamp — a double claim cannot pay twice
<code>grantReward()</code>	Create <code>RewardEvent</code> (unique <code>(user, kind, refId)</code>) $\rightarrow$ <code>credit()</code> $\rightarrow$ backfill <code>txHash</code>
<code>anchor()</code>	Never throws. Returns <code>{ index: -1, hash: '', txs: [] }</code> on failure

Operational errors use `EngineError` with a `code` (`INSUFFICIENT` , `NO_WALLET` , `BAD_AMOUNT` , `BAD_PARAM` , `SAME_WALLET` , `NO_KEY`). Routes map any `error.code` to **400** and everything else to **500**.

10.6 Idempotency — the reward safety net



The uniqueness is in the **database**, so a double click, a retried request or two racing tabs cannot mint twice. Route-level checks could all be bypassed by concurrency; this cannot.

10.7 Pure rules — `rules.js`

Every "smart contract" decision is a pure function with no I/O:

Function	Contract
<code>tallyProposal(votes, params, now)</code>	One vote per wallet. <code>passed = quorumReached && forWeight > againstWeight</code> . Ties fail.
<code>checkinStreak(days, todayKey)</code>	Consecutive run ending today or yesterday; else 0
<code>checkinReward(params, streak)</code>	Base + step × min(streak, 10) — the step is capped
<code>computeStakeReward(staked, apyPct, elapsedMs)</code>	<code>amount × apy/100 × days/365</code> ; 0 for any non-positive input
<code>escrowSplit(total, feePct)</code>	<code>{ fee, proceeds }</code> , <code>feePct</code> clamped to [0, 100]
<code>disputeOutcome(votes, jurors)</code>	Quorum, not unanimity . Open jury needs 3 agreeing; a closed panel needs <code>max(1, ceil(size/2))</code> . Non-juror votes ignored when a panel exists. A tie pays nobody.
<code>eligibleAchievements(stats, ownedKinds)</code>	Server-side eligibility from real activity

10.8 The DAO parameter contract

`DEFAULT_PARAMS` in `server/models/Web3.js` is the ABI. The frontend's updatable list, the proposal validator and the engine all read from it.

```
Add a parameter to DEFAULT_PARAMS
  existing config rows are backfilled automatically on read
  it appears in GET /dao/config -> updatable
  and becomes proposable
No migration needed.
```

`setParam` uses a `hasOwnProperty` guard against prototype-chain keys, and `assertParamValue` rejects non-finite, negative, `> 1e9` , and constrains `daoVotingDays` to 1–365.

10.9 The auth model

`routes/web3/index.js` registers exactly three public endpoints **before** `protect` :

Method	Path	Why public
GET	/verify/:code	A QR code on a bottle must work with no account
GET	/verify/:code/qr?base=	QR data URL
GET	/share/:token	A clinician must read a shared profile with no account

Everything else is behind `protect` → `userOnly` → `web3PlanGate` → the sub-routers. `userOnly` returns 403 for `role === 'admin'`.

If you add a router, mount it **inside** that group so the guards apply.

10.10 Frontend integration rules

- `src/api/web3.js` is a **thin client** — one function per endpoint, no state, no business rules. Add the function there before touching a panel.
- Panels in `src/Components/Web3Panels/` are **default exports with no props**.
- All styling is `Web3.css` with a `w3-` **prefix**. Do not introduce unprefixed classes.
- Mount-time fetches use the established lint convention: `jsx useEffect(() => { load(); /* eslint-disable-line react-hooks/exhaustive-deps, react-hooks/set-state-in-effect -- intentional initial load on mount */ }, []);`
- `/web3`, `/marketplace`, `/governance` are `ProtectedRoute` **plus** a plan gate, so a sub-tier user never triggers the panels' fetches.
- `/verify`, `/verify/:code` and `/share/:token` are **intentionally public**. Never wrap them in `ProtectedRoute`.

11. Rate limiting and abuse control

11.1 Limiters

Limiter	Window	Max (local / remote)	Protects
<code>requestFloodGuard</code>	60 s	12000 / 1200	every <code>/api</code> path incl. health and the JSON 404
<code>sessionLimiter</code>	15 min	1500 / 400	<code>/api/auth/me</code> , <code>/api/subscription</code>
<code>authLimiter</code>	15 min	200 / 20	<code>/api/auth</code> ; skips session reads
<code>recommendLimiter</code>	10 min	15	<code>/api/recommend</code> — protects AI quota
<code>aiLimiter</code>	10 min	300 / 60	<code>/api/polish</code> , <code>/api/supplement-detail</code>
<code>userLimiter</code>	1 min	600 / 120	assessment, chat, dashboard, insights, notifications, security, web3
<code>adminLimiter</code>	1 min	300 / 60	<code>/api/admin</code>

`authLimiter` and `adminLimiter` use `limitReachedHandler` , which **escalates the lockout ladder**.
`userLimiter` deliberately does **not** — normal feature traffic must never climb the brute-force ladder.

11.2 Two layers of flood defence

```
requestFloodGuard  global per-IP envelope that METERS but never escalates
credential tripwire 20 distinct failed accounts per IP / 15 min --> 5-minute cooldown,
                    in a bucket SEPARATE from the hard stop
```

They are separate on purpose. One is capacity; the other is attack detection.

11.3 Body metering runs before parsing

```
Content-Length pre-check --> 413 immediately, body never read
in-flight byte budget    --> 503 + Retry-After when the global envelope is full
both mounted BEFORE express.json()
```

A security ceiling is also a capacity number: behind a shared NAT every user is one public address, so a value tuned for a home connection locks out a whole school. Unset, empty or nonsense values fall back to the documented default — **a typo must never switch rate limiting off**.

11.4 Owner-scoped reads

Every `find` pairs the object id with `req.user._id`. A miss returns **404**, **never 403** — 403 would confirm that the id exists.

12. Testing

12.1 Commands

```
# Server -- 1006 tests
cd server && npm test

# Frontend -- 637 tests
cd my-react-app && npm test

# Lint -- must stay at 0 errors
cd my-react-app && npm run lint

# Build -- must succeed
cd my-react-app && npm run build

# Syntax check across the whole server tree
cd server && npm run check

# Live HTTP suites (need a server / database)
cd server && npm run test:flows
cd server && node test-web3-flows.js
```

12.2 Contract tests that fail when two truths drift

These are the highest-value tests in the repo:

Contract	Guards
<code>features.test.js</code>	the frontend feature registry against the server entitlements
<code>password-policy-agreement.test.js</code>	the client password rules against the shipped server policy
<code>optimizeDeps</code> completeness	a dependency first reached through a dynamic import triggers a re-optimize that invalidates every module URL the open page holds
<code>plan-catalogue.test.js</code>	the absence of the removed Team band
<code>verifyDevDeps.mjs</code>	a stale <code>/node_modules/.vite/deps/*.js?v=<hash></code> → 504. Unit tests, eslint and <code>vite build</code> all pass with that dep missing

12.3 Test-database safety

`Test File/testDbGuard.js` keys on a **separate URI**, never on a database name, because the application's `MONGO_URI` carries no database in its path and silently resolves to the driver default literally named `test` on the live cluster. With nothing configured the suite **skips** rather than fails.

12.4 Live HTTP suites

Script	Covers
<code>test-web3-flows.js</code>	all 20 Web3 features over real HTTP (81 checks)
<code>test-subscription-flows.js</code>	purchase, plan requests, downgrade, cancel, the SSE push
<code>test-subscription-admin.js</code>	the admin journey incl. day grants and "restore original"
<code>test-session-flows.js</code>	parallel accounts, displacement, revocation, no-expiry tokens
<code>test-priority-override-flows.js</code>	live admin Priority raise/release (needs <code>MONGO_TEST_URI</code>)
<code>test-ddos-resilience.js</code>	loopback-only flood/stress test; refuses any non-loopback host
<code>glitch-hunt.js</code> / <code>glitch-hunt-web3.js</code>	adversarial probes: IDOR, injection, prototype chain, race conditions

Never run two `test-web3-flows.js` concurrently. The second run's cleanup deletes the first run's `web3-smoke-*` users mid-flight and the victim fails with confusing `400` s. If a run fails oddly, check for a concurrent run before hunting a code bug.

13. Extension playbooks

13.1 Adding a Web3 feature

1. **Model** (if needed) in `server/models/Web3.js`. Never store personal data that will be anchored — anchor a digest.
2. **Pure rules** in `blockchain/rules.js` + unit tests in `Test File/web3.test.js`.
3. **Route** in the matching `routes/web3/*.js`. Validate lengths explicitly (truncate with `.slice()` or reject with 400 — never let a schema `maxlength` surface as a 500), mutate through `engine`, then `engine.anchor(...)`, and map `error.code` → 400.
4. **Idempotency** for any payout: a unique index plus `engine.grantReward`.
5. **API client** function in `src/api/web3.js`, then a panel and a `w3-`-prefixed section.
6. **Smoke checks** for the happy path, the replay/idempotency path, and one authorization path (a party cannot vote, an admin is rejected, own content cannot be upvoted).

7. Run the full matrix in §12.1.

13.2 Adding an entitlement

1. Add the key to `FEATURES` in `server/utils/entitlements.js` with a `minTier`.
2. Mirror it in `my-react-app/src/subscription/features.js`.
3. `features.test.js` now guards the pair.
4. Gate the route with `requireFeature('yourKey')`.
5. Render a `PlanLockedCard` in the client rather than hiding the feature.

13.3 Adding a DAO parameter

Add it to `DEFAULT_PARAMS`. Existing config rows backfill on read; it appears in `GET /dao/config → updatable` and becomes proposable. **No migration.**

13.4 Routing a feature to a different AI provider

Edit `AI_ROUTES` in `server/utils/aiRouter.js`. Update `utils/aiProviders.js#providerUsage()` only if you add a *new* provider — it reads the table lazily so a provider card can never say "no route calls this" while routes to it exist.

13.5 Adding a page

`lazy(() => import(...))`, register the route in `App.jsx` inside the shared `<Suspense>`, pick the right guard (`ProtectedRoute` / `PublicOnlyRoute` / `AdminProtectedRoute`), and add its nav entry. If the page needs a dependency first reached through a dynamic import, add it to `optimizeDeps.include` — the completeness test will tell you.

14. Operations and troubleshooting

14.1 Boot refusals

The server **refuses to start** rather than run insecurely. Each failure mode below is invisible until a real user is affected:

Refusal	Why
<code>JWT_SECRET</code> missing or the shipped placeholder	every session and wallet key envelope derives from it
<code>JWT_SECRET</code> shorter than 32 chars	entropy
<code>ALLOW_DEV_OTP_RESPONSE=true</code> with <code>NODE_ENV=production</code>	would leak OTPs in API responses
<code>PUBLIC_WEB_URL</code> unset in production	password-reset link origin
trusted-origin allowlist empty in production	CORS + CSRF
no TOTP encryption key in production	a TOTP seed cannot be hashed, so anyone who can read the database could mint valid codes forever
TOTP key would be derived from <code>JWT_SECRET</code> in production	couples two unrelated secrets
<code>WEBAUTHN_RP_ID</code> / origins unset in production	ceremonies cannot be bound
admin token lifetime $\leq$ admin idle window	logs a working administrator out mid-task
<code>MONGO_URI</code> unset	nothing to connect to

14.2 Common symptoms

Symptom	Cause / fix
<code>EADDRINUSE :::5000</code>	Another API instance is running. The crash guard logs and keeps the process alive <i>without</i> a listener — kill the stale process. Verify with <code>Get-NetTCPConnection -LocalPort 5000 -State Listen</code> .
Health check fails right after start	Atlas can take 30–60 s to connect. The process is alive but silent until the first successful connect. Retry <code>/api/health</code> for a minute and look for <code>Connected to MongoDB</code> — an empty log with a live PID means <i>still connecting</i> , not crashed.
<code>privateKeyEnc: Cast to string failed</code>	The envelope must be an object column <code>{iv, ct, tag}</code> , never a <code>String</code> . The schema was edited back.
<code>E11000 ... tokenId_1</code> on achievements	A lost mint race. Treat it as "already owned", recompute a serial — not a 500.
<code>Insufficient WELL balance</code> unexpectedly	A concurrent purge deleted the wallet mid-request (see the smoke warning), or the DAO genuinely spent it. Check <code>GET /wallet</code> and the history.
Juror pool looks wrong	Orphaned wallets from deleted accounts. Clean up with the same rule as the smoke test: <code>isSystem != true</code> and no live user.
Chain height not growing	Anchors are best-effort; a persistent failure logs <code>[web3 anchor] ...</code> . Check Mongo connectivity and <code>CHAIN_DIFFICULTY</code> .
A page renders but a fetch says "aborted"	Usually the tab navigated away mid-request (logout/redirect). Reproduce in a fresh tab before investigating.
Frontend lint parse error after a page edit	A duplicate top-level declaration — <code>Identifier 'X' has already been declared</code> . Remove the stale copy.
<code>Failed to fetch dynamically imported module ... /node_modules/.vite/deps/<pkg>.js?v=<hash></code>	A stale dev dependency cache. <code>npm run dev</code> runs <code>healDepsCache.mjs</code> first, which repairs it. <code>verifyDevDeps.mjs</code> is the check that can actually see this failure.

14.3 Security-critical environment variables

Variable	Why it matters
TOTP_ENCRYPTION_KEY	Seals TOTP seeds. Generate <code>openssl rand -base64 32</code> . <code>TOTP_ENCRYPTION_KEY_PREVIOUS</code> supports <code>kid=base64</code> rotation.
WEBAUTHN_RP_ID	A bare domain — no scheme, no port, no path
WEBAUTHN_ORIGIN	Comma-separated <code>https</code> origins allowed to run a ceremony; also trusted by CORS
WEB_ALLOWED_ORIGINS	Origins allowed to make credentialed requests. Never a wildcard
PUBLIC_WEB_URL	The address used in emailed links. Required in production
ALLOW_LAN_ORIGINS	Development only — trust <code>http://192.168.x.x:port</code> so a phone can reach a dev backend
TRUST_PROXY	<code>true</code> only behind a proxy you control. Off by default so <code>req.ip</code> cannot be spoofed via <code>X-Forwarded-For</code>

Rotating `JWT_SECRET` makes existing `privateKeyEnc` values undecryptable — the wallet key envelope is keyed from it. Plan a re-encryption migration before rotating in production.

14.4 Environment loading

`index.js` requires `utils/envFile.reloadFromDisk()`, **never bare** `dotenv`, so a reload can distinguish an OS-provided variable from one that only exists because `.env` defined it.

`node --test` gives each test file **its own process**, so harness files must load `.env` themselves.

14.5 Administrator credential tooling

Two deliberately separate code paths, not one with a flag — precisely so the recurring channel can never grow a password field.

Command	Carries	Schedule
<code>npm run notify-admins</code>	alias only, secret-free	Safe to re-run and automate
<code>npm run send-admin-credentials -- --password Alias: '...'</code>	alias, password, TOTP seed	One-time only. Never scheduled

Both support `--dry-run`. The hand-off refuses to send a partial set: an administrator with an address but no seed or no password cannot finish signing in, so the script reports every unusable alias and exits 1 rather than mailing half a credential set. Split `--password` on the **first** colon, so a password may itself contain colons.

****SuppliWise Developer Manual**** — see ``README.md`` for setup, ``BLOCKCHAIN_FEATURES.md`` for the feature → code map, ``SECURITY_AUDIT_REPORT.md`` for the audit, and ``docs/USER_MANUAL.md`` for the end-user guide.